

FINAL YEAR PROJECT · 2026

# Virtual Fit.

Try it on before you buy it — a browser-based,  
body-aware virtual fitting room.

100% in-browser body detection

AI fashion assistant

Full storefront + admin

Privacy-first

## THE PROBLEM

# Clothes shopping online is a guess.

## 30–40%

of online apparel is returned — **fit and sizing are the #1 reason.**

## Inconsistent

sizes across brands make charts useless — a "M" differs everywhere.

## Existing tools

need server-side ML pipelines, GPUs, or send your **photos to the cloud.**

→ Small stores can't afford try-on tech, and shoppers can't verify fit **before** checkout.

# A fitting room that lives in the browser tab.

## WHAT THE SHOPPER DOES

- Uploads or snaps **one photo** (shoulders to hips)
- Picks a garment from the live catalog
- Sees it fitted to **their body instantly**
- Fine-tunes scale, offset, length, rotation
- Compares pieces side-by-side, adds to cart

## WHAT MAKES IT DIFFERENT

- Pose detection runs **on-device** (WebAssembly) — photos never leave the browser for processing
- No GPU, no ML servers — runs on a **\$5 VPS**
- Plain-language AI assistant: *"make it slightly oversized"*
- Complete e-commerce loop: accounts, cart, orders, admin

# From photo to fitted — in one second, client-side.

**1 Photo intake**  
Upload, drag-drop, live camera (with timer) or native mobile camera. Re-encoded & downscaled to  $\leq 1400\text{px}$  in the browser.

**2 Body landmark detection**  
MediaPipe Pose Landmarker (33 landmarks) via self-hosted WASM — shoulders, hips, torso visibility scored for confidence.

**3 Anchor-based garment mapping**  
Each garment ships with exact anchor geometry (shoulder & hem points). A geometric engine scales, skews and rotates it onto the detected torso.

**4 Interactive composite**  
Canvas rendering with zoom/pan, before/after toggle, skeleton overlay, save or download — all without another server round-trip.

# More than a demo — a working store.

## FITTING ROOM

- Body-aware fitting
- Fine-tune sliders
- Before / after
- Compare 2 garments

## ASSISTANT

- Plain-language fit edits
- Occasion styling
- Recommends garments
- Works with no API key

## STOREFRONT

- Catalog + product pages
- Cart drawer
- Checkout + orders
- Accounts & history

## ADMIN

- Garment uploads
- Categories & orders
- User management
- Status workflow

**Demo catalog:** 15 procedurally-drawn garments across 6 categories — each with per-garment anchor metadata generated from its drawing geometry, so fitting is pixel-accurate.

# Talk to your wardrobe.

## WHAT IT UNDERSTANDS

- **"Make it slightly oversized"** → widens scale  $\pm$
- **"Something casual for university"** → filters by occasion & formality
- **"Compare the shirt and the jacket"** → opens comparison
- **"Why is this more formal?"** → explains catalog metadata

## DUAL-ENGINE DESIGN

- **LLM path** — OpenRouter (free-tier model) returns structured JSON commands validated with Zod
- **Local rules engine** — deterministic NLU fallback when no key / offline; the app never loses the feature
- Commands map to a typed **apply()** pipeline — the model can only do what the UI allows

# Boring where it should be, clever where it counts.

## FRONTEND

- Next.js 14 (App Router, standalone)
- React 18 server components
- Tailwind CSS, responsive + mobile chat sheet
- Canvas 2D fitting engine

## ON-DEVICE ML

- MediaPipe Tasks Vision (WASM)
- Pose Landmarker Lite (~6 MB, self-hosted)
- SIMD + no-SIMD builds shipped
- Zero third-party CDN dependency

## BACKEND & DATA

- API routes with Zod validation
- Prisma ORM → Supabase Postgres
- sharp asset pipeline (WebP)
- Docker → Dokploy on AWS Lightsail

**Data model:** User · Category · Product (with anchor metadata) · Order · OrderItem — 5 tables, fully relational with FK constraints.

# Your body, your device.

- **Pose detection runs in the browser** — WASM on the user's CPU, not a GPU cluster
- Photos are **never uploaded for processing**
- Saved results are opt-in and **auto-expire** (24h default) via a cleanup sweeper
- Upload validation: type, size (10 MB), dimension caps, rate limits

- **bcrypt (cost 12)** password hashing
- **HMAC-SHA256 signed**, httpOnly, SameSite=Lax session cookies — protocol-aware Secure flag
- Admin gate on a separate signed cookie; all admin APIs guarded
- Zod schema validation on every endpoint; asset paths whitelisted (no traversal)

# Featherweight on purpose.

**<110kB**

first-load JS per route

**~1s**

photo → fitted result (on-device)

**512MB**

VPS runs the whole stack

**\$0**

AI cost — free-tier LLM + local  
fallback

**Ops:** Docker image self-heals — schema sync on boot, seeded demo catalog, persistent asset volume, health endpoint + container healthcheck. Deploys are one click via Dokploy.

# Where this goes next.

## PRODUCT

- Real garment photography via admin uploads
- Multi-layer outfits (shirt + jacket stacks)
- Size recommendation from detected proportions
- Live AR try-on via `getUserMedia` loop

## PLATFORM

- Supabase Auth (Google login) alongside cookie auth
- Supabase Storage for garment assets
- Payments (Stripe test mode)
- i18n + accessibility audit pass

# Questions you might ask.

## Why geometric fitting instead of a generative ML try-on?

Garment-accurate generative try-on needs diffusion models, GPUs, and sends photos to servers — the opposite of this project's privacy goal. Anchor-based geometry is deterministic, instant, explainable, and runs offline. It's a deliberate trade: realism vs. privacy + cost.

## How accurate is the fit, honestly?

It maps 4 anchors (both shoulders, both hems) onto detected landmarks with per-garment length/width factors, then lets the user fine-tune. It reproduces proportions faithfully (shoulder span, torso length) but does not simulate fabric drape — stated openly as a limitation.

## Where does pose detection run, and with what model?

MediaPipe Pose Landmarker Lite (float16, ~6MB), executed as WebAssembly on the user's CPU. The model and WASM runtime are self-hosted in /public — no CDN, works offline after first load. A confidence score gates bad photos with actionable hints.

## Why SQLite first, then Supabase Postgres?

SQLite gave zero-setup local development. Supabase (managed Postgres) adds concurrent writes, daily backups, and a dashboard for the deployed app. Prisma kept all query code identical — only the datasource changed.

## Why custom auth instead of NextAuth / OAuth?

The requirement was a self-contained system demonstrating the full flow: bcrypt hashing, HMAC-signed httpOnly cookies, sessions, role gating. Supabase Auth is on the roadmap as an additional provider.

## How does the AI assistant stay safe / functional without an API key?

LLM output is constrained to a Zod-validated JSON command schema applied through a typed pipeline — the model can only trigger actions the UI already allows. With no key, a deterministic local NLU handles the same intents, so demos never break.

# Questions you might ask. (cont.)

## What happens to user photos?

Processing is 100% client-side. A photo reaches the server only when the user explicitly saves a result — stored with a 24h TTL and swept by a retention job. Camera access requires HTTPS; uploads are type/size/rate limited.

## How would it scale to real traffic?

The heavy compute is in each visitor's browser, so the server only serves the app + catalog. Bottleneck order: asset volume I/O → Postgres pool → node process. Fixes: object storage for assets, pgbouncer pooling, horizontal replicas behind the proxy — all standard.

## What was the hardest engineering problem?

Making anchor geometry look right across bodies: deriving per-garment hem/shoulder anchors from drawing coordinates, then composing scale, skew and rotation without visible artifacts. Runner-up: production runtime behavior — cookies and camera APIs over plain HTTP vs HTTPS.

## How was it tested?

Zod-validated API contracts, e2e scripts (scripts/e2e-\*.mjs) covering register → cart → order flows, a /api/dev/selftest endpoint, catalog validation and asset integrity scripts, plus live end-to-end verification of the Docker deployment path.

## What did it cost to build and run?

Build: \$0 (all open-source). Run: a ~\$5–10/mo Lightsail VPS + Supabase free tier + optional free OpenRouter key. No paid services are required.

## What's fake vs. real in the demo?

Real: accounts, sessions, catalog CRUD, cart, orders, admin, fitting engine, assistant. Simulated by design: payment at checkout and shipping — clearly labelled, since handling real money wasn't in scope.

THANK YOU

**See it on your body,  
not on a mannequin.**

Live demo · [vfit.rukaiya.tech](https://vfit.rukaiya.tech) · Source · [github.com/Rukaiya-Sultana/virtual-fit](https://github.com/Rukaiya-Sultana/virtual-fit)

Rukaiya Sultana

[rukaiyam70@gmail.com](mailto:rukaiyam70@gmail.com)

2026